

Síntesis de frecuencia de alta precisión con PLL_SYS y PIO en RP2040

Optimización para 7,074 / 14,074 / 28,074 MHz y corrección de Bresenham

Proyecto:

Firmware PixiePico

Plataforma: RP2040 Zero – Pico SDK – PIO

Fecha: 1 de septiembre de 2026

Autor: Dr. Pedro E. Colla (LU7DZ)

Resumen del problema

Para una salida cuadrada simple utilizando una arquitectura rp2040 requiere dos estados lógicos por período. Con el divisor PIO en formato **16.8**, la frecuencia ideal de un programa de dos estados es

$$f_{\text{out}} = \frac{128 f_{\text{sys}}}{N},$$

donde N es entero y representa 256 veces el divisor completo. La búsqueda exhaustiva sobre todos los PLL_SYS realmente alcanzables entre 125 y 250 MHz no logra un error estático menor que 1 Hz en los tres ejemplos. Los mínimos son 5,5266 Hz, 25,5548 Hz y 50,4202 Hz. Sin embargo, alternando duraciones adyacentes con un acumulador de error, el período medio teórico puede hacerse igual al objetivo.

La precisión teórica (algorítmica) no equivale a precisión real: para garantizar ± 1 Hz a 28,074 MHz la referencia de 12 MHz debe estar calibrada mejor que 0,036 ppm, condición que el reloj de una placa rp2040 no cumple y cuya falta de precisión dominará el error final aunque la precisión media sea cero.

1. Alcance y supuestos

Se intenta desarrollar una única salida de onda cuadrada. Un período completo contiene un estado alto y uno bajo; por ello el factor de estados es $S = 2$. El reloj de sistema puede someterse a "overclock" pero por consideraciones prácticas se limita el intervalo al rango 125 a 250 MHz. Por otra parte, no todas las frecuencia de clock son viables puesto que éstas se logran mediante divisores tales que puedan producir el PLL_SYS del RP2040 deseado a partir de XOSC=12 MHz y con la ruta usada por `set_sys_clock_pll()` y `check_sys_clock_hz()`.

1.1. Restricciones del espacio discreto

Elemento	Restricción usada
XOSC	12000000 Hz
PLL reldiv	Fijo según PLL_SYS_REFDIV del SDK; no es posible asignar reldiv arbitrarios que <code>set_sys_clock_pll()</code> no aplica.
FBDIV	Entero; se enumera 16–320 y se filtra por VCO válido.
VCO	750–1600 MHz.
POSTDIV1/2	1–7, con $POSTDIV1 \geq POSTDIV2$.
PLL_SYS	125–250 MHz y división exacta del VCO.
Divisor PIO	$D = d_{\text{int}} + d_{\text{frac}}/256 = N/256$; $N \geq 256$. El hardware utiliza formato 16.8.
Forma de onda	Dos estados PIO por período: alto y bajo.

El SDK busca configuraciones con un VCO válido y postdivisores enteros; `check_sys_clock_hz()` sólo acepta una frecuencia si el resultado del PLL es exacto. Esta validación debe ser tomada en cuenta en la solución pues caso contrario el estado del sistema corre el riesgo de quedar inestable y otros subsistemas funcionar incorrectamente, por ejemplo el sistema de soporte a USB.

1.2. Hipótesis y límites

El máximo alcanzado consistentemente en las placas disponibles fue 250 MHz, a la cual el hardware funcionó correctamente (para un clock nominal sin "overclock" de 125 MHz. 270 MHz funciona en algunas placas y produce fallas de USB en otras. Es por lo tanto necesario limitar rango de búsqueda a 250 MHz. Este rango parece ser seguro para las unidades probadas, aunque no está en la especificación del rp2040 (que como se dijo trabaja con un clock de 125 MHz). Los cálculos suponen un XOSC ideal de 12 MHz; la tolerancia real debe medirse y compensarse.

2. Modelo matemático

2.1. Reloj del PLL

De acuerdo al método estándar propuesto por el SDK como referencia $f_{\text{ref}} = 12 \text{ MHz}$, un candidato de sistema es:

$$f_{\text{sys}} = \frac{12000000 \text{ FBDIV}}{\text{POSTDIV1 POSTDIV2}}.$$

Sólo se conserva el candidato si $\text{VCO} = 12 \text{ MHz} \cdot \text{FBDIV}$ está dentro del rango del PLL, los postdivisores son válidos, $f_{\text{sys}} \in [125, 250] \text{ MHz}$ y `check_sys_clock_hz()` confirma la misma configuración.

2.2. Divisor PIO y salida de dos estados

El divisor 16.8 del PIO se representa mediante el entero $N = 256D$. Para dos instrucciones o estados equivalentes por período:

$$D = \frac{N}{256}, \quad f_{\text{out}} = \frac{f_{\text{sys}}}{2D} = \frac{128 f_{\text{sys}}}{N}.$$

Para cada candidato PLL:

$$N^* = \frac{128 f_{\text{sys}}}{f_{\text{objetivo}}}, \quad N_0 = \lfloor N^* \rfloor, \quad N_1 = N_0 + 1.$$

Basta evaluar N_0 y N_1 porque f_{out} es monótona en N . La optimización global escoge la combinación que minimiza

$$\left| \frac{128 f_{\text{sys}}}{N} - f_{\text{objetivo}} \right|.$$

2.3. Pseudocódigo del algoritmo

```
best.error = infinito
para cada FBDIV permitido:
    VCO = 12 MHz * FBDIV
    si VCO no es valido: continuar
    para POSTDIV1=1..7 y POSTDIV2=1..POSTDIV1:
        si VCO no divide exactamente: continuar
        f_sys = VCO/(POSTDIV1*POSTDIV2)
        si f_sys no pertenece a [125,250] MHz: continuar
        si check_sys_clock_hz(f_sys) falla: continuar
        N_ideal = 128*f_sys/f_objetivo
        para N en {floor(N_ideal), ceil(N_ideal)}:
            si N < 256: continuar
            f_out = 128*f_sys/N
            error = f_out - f_objetivo
            conservar si |error| mejora best
```

3. Resultados parciales

Objetivo	PLL_SYS	PLL (VCO; FB; P1/P2)	N y divisor 16.8	Salida	Error
7,074 MHz	180,000 MHz	900 MHz; 75; 5/1	$N = 3257, D = 12 + \frac{185}{256}$	7073994.473442 Hz	-5.526558 Hz
14,074 MHz	163,500 MHz	1308 MHz; 109; 4/2	$N = 1487, D = 5 + \frac{207}{256}$	14073974.445192 Hz	-25.554808 Hz
28,074 MHz	130,500 MHz	1044 MHz; 87; 4/2	$N = 595, D = 2 + \frac{83}{256}$	28073949.579832 Hz	-50.420168 Hz

La tabla es el resultado de una búsqueda exhaustiva sobre 246 frecuencias PLL configurables distintas. Bajo los supuestos definidos no existe otra combinación estática PLL/divisor 16.8 con menor error.

3.1. Caso 7074000 Hz

$$\text{VCO} = 12 \text{ MHz} \cdot 75 = 900 \text{ MHz}, \quad f_{\text{sys}} = \frac{900}{5 \cdot 1} = 180 \text{ MHz}.$$

$$N^* = \frac{128 \cdot 180000000}{7074000} = \frac{1280000}{393} = 3256.997455470738.$$

$$N_0 = 3256, \quad D_0 = 12 + \frac{184}{256}, \quad f_0 = 7076167.076167076 \text{ Hz}, \quad e_0 = +2167.076167076 \text{ Hz},$$

$$N_1 = 3257, \quad D_1 = 12 + \frac{185}{256}, \quad f_1 = 7073994.473441818 \text{ Hz}, \quad e_1 = -5.526558182 \text{ Hz}.$$

3.2. Caso 14074000 Hz

$$\text{VCO} = 12 \text{ MHz} \cdot 109 = 1308 \text{ MHz}, \quad f_{\text{sys}} = \frac{1308}{4 \cdot 2} = 163.5 \text{ MHz}.$$

$$N^* = \frac{128 \cdot 163500000}{14074000} = \frac{10464000}{7037} = 1486.997299985789.$$

$$N_0 = 1486, \quad D_0 = 5 + \frac{206}{256}, \quad f_0 = 14083445.491251683 \text{ Hz}, \quad e_0 = +9445.491251682 \text{ Hz},$$

$$N_1 = 1487, \quad D_1 = 5 + \frac{207}{256}, \quad f_1 = 14073974.445191661 \text{ Hz}, \quad e_1 = -25.554808339 \text{ Hz}.$$

3.3. Caso 28074000 Hz

$$\text{VCO} = 12 \text{ MHz} \cdot 87 = 1044 \text{ MHz}, \quad f_{\text{sys}} = \frac{1044}{4 \cdot 2} = 130.5 \text{ MHz}.$$

$$N^* = \frac{128 \cdot 130500000}{28074000} = \frac{2784000}{4679} = 594.998931395597.$$

$$N_0 = 594, \quad D_0 = 2 + \frac{82}{256}, \quad f_0 = 28121212.121212121 \text{ Hz}, \quad e_0 = +47212.121212121 \text{ Hz},$$

$$N_1 = 595, \quad D_1 = 2 + \frac{83}{256}, \quad f_1 = 28073949.579831932 \text{ Hz}, \quad e_1 = -50.420168067 \text{ Hz}.$$

4. Corrección temporal tipo Bresenham

Es claro que la tolerancia del método es adecuada para algunas aplicaciones (por ejemplo CW) pero no para otras (por ejemplo FT8). Se busca un método de compensación calculando una frecuencia media a partir de la utilización del método de Bresenham. Si se alternan $N_0 = \lfloor N^* \rfloor$ y $N_1 = N_0 + 1$, el parámetro correcto que debe promediarse es N , porque el período es lineal en N :

$$T(N) = \frac{N}{128f_{\text{sys}}}.$$

Sea $r = N^* - N_0$. Se utiliza N_1 durante la fracción r de los períodos y N_0 durante $1 - r$. Entonces:

$$\bar{N} = (1 - r)N_0 + rN_1 = N^*, \quad \bar{T} = \frac{1}{f_{\text{objetivo}}}.$$

El error medio teórico es cero. No se deben promediar las frecuencias $f(N_0)$ y $f(N_1)$, porque la magnitud físicamente acumulada es el período o la fase.

Objetivo	N_0 corto	N_1 largo	Uso exacto por patrón	Longitud	Repetición
7,074 MHz	3256	3257	N_1 : 392; N_0 : 1	393 períodos	18.000 Hz
14,074 MHz	1486	1487	N_1 : 7018; N_0 : 19	7037 períodos	2.000 Hz
28,074 MHz	594	595	N_1 : 4674; N_0 : 5	4679 períodos	6.000 Hz

Para 7,074 MHz,

$$N^* = 3256 + \frac{392}{393}.$$

La secuencia contiene 392 períodos con $N = 3257$ y uno con $N = 3256$. Bresenham distribuye el período corto sin acumular más de una unidad de error del acumulador.

4.1. Acumulador entero

```
// r = r_num/r_den; devuelve N0 o N1 para el proximo periodo
uint32_t next_N(fracn_t *s) {
    s->acc += s->r_num;
    if (s->acc >= s->r_den) {
        s->acc -= s->r_den;
        return s->N1; // periodo ligeramente mas largo
    }
    return s->N0; // periodo ligeramente mas corto
}
```

La secuencia exacta es periódica y genera espurias a la frecuencia de repetición indicada y sus armónicos. Si el espectro exige menor concentración tonal, puede emplearse difusión de error de orden superior o aleatorización controlada, conservando el número de N_0/N_1 en ventanas largas.

5. Implementación en RP2040 PIO

5.1. Limitación

Una state machine PIO no puede escribir su propio registro CLKDIV. Por tanto, alternar directamente N_0/N_1 no puede implementarse con instrucciones PIO puras. Requiere cooperación de CPU/DMA o un generador temporal PIO rediseñado que alterne duraciones de estado. Cambiar CLKDIV desde una interrupción por cada período sería inviable a 7–28 MHz y produciría jitter en magnitudes no deseables.

5.2. Arquitectura: PIO + DMA + patrón de tiempo

La alternativa posible es no tratar de reescribir CLKDIV en cada ciclo. Se fija un reloj de la state machine y se entrega por DMA un flujo de decisiones corto/largo generado por el acumulador Bresenham. Cada bit determina si el siguiente semiperíodo consume K o $K + 1$ ticks de la state machine. La PIO usa side-set para conmutar el pin y una rama balanceada para insertar el tick opcional. El solver debe incluir los ciclos de `out/jmp/nop` en K ; no puede reutilizar sin cambios la fórmula ideal de dos instrucciones.

Con autopull de 32 bits, incluso 56,148 millones de decisiones de semiperíodo por segundo requieren aproximadamente 7,02 MB/s de lectura DMA, una carga razonable desde SRAM. El patrón debe residir en SRAM y usar un anillo DMA; no debe leerse desde flash mientras otras funciones escriben XIP.

```
; Script PIO un bit por semiperíodo
.program bresenham_square
.side_set 1
.wrap_target
    out x, 1        side 1 [BASE_H] ; bit=1 agrega un tick
    jmp !x high_ok side 1
    nop             side 1
high_ok:
    out x, 1        side 0 [BASE_L]
    jmp !x low_ok  side 0
    nop             side 0
low_ok:
.wrap
; autopull=32; DMA alimenta TX FIFO desde un buffer circular
```

Este fragmento es por ahora un esqueleto: `BASE_H` y `BASE_L`, junto con los ciclos de las ramas, deben resolverse para cada banda de frecuencia, verificando que ambos caminos tengan exactamente K o $K + 1$ ticks. La salida cambia sólo en instrucciones con side-set, por lo que la fase queda acotada a un tick de la state machine.

5.3. Alternativa, dither del registro CLKDIV

También puede escribirse una tabla de N_0/N_1 en `PIO_SMx_CLKDIV` mediante DMA sincronizado a límites de período. Conserva un jitter menor, pero exige un mecanismo adicional de pacing y sincronización porque la state machine no genera por sí sola un DREQ de fin de período. A 28 MHz, transferir un word por período consume cerca de 112 MB/s y queda demasiado próximo al límite práctico; por eso no es la opción recomendada salvo que se use compresión por rachas y sincronización.

6. Solver y flujo de firmware

Etapas	Responsabilidad	Fallo seguro
1. solve	Enumera PLL + divisor; no toca hardware.	Devuelve false si no hay solución.
2. validate	<code>check_sys_clock_hz</code> ; límites VCO/PLL/PIO/error.	Rechaza el candidato antes de aplicarlo.
3. apply	Aplica PLL antes de USB y verifica <code>clock_get_hz</code> .	Vuelve a 250 MHz o reinicia.
4. USB	<code>TUDstart</code> sólo con reloj estable.	No reconfigura PLL con USB montado.
5. generate	PIO fijo o PIO+DMA Bresenham.	Detiene la salida ante underrun.

Para un cambio de frecuencia en ejecución, el solver primero comprueba si el PLL actual permite una solución aceptable cambiando sólo el patrón o divisor. Si exige otro PLL, se guarda la frecuencia en watchdog scratch y se reinicia; reprogramar PLL_SYS después de enumerar dispositivos como parte del subsistema TinyUSB (que opera concurrentemente) probablemente genere inestabilidades en el firmware.

6.1. Criterio de selección

El primer criterio es error medio ≤ 1 Hz. Entre candidatos que cumplen, se minimizarán, en este orden: cambio respecto del PLL actual, tasa de correcciones Bresenham, longitud y tonalidad del patrón, jitter pico y consumo. Escoger únicamente el menor error estático puede producir una secuencia de corrección espectralmente peor.

6.2. Validaciones de ejecución

Prueba	Condición de aceptación
PLL aplicado	<code>clock_get_hz(clk_sys)</code> coincide con la solución; <code>clk_usb</code> permanece en 48 MHz.
Divisor	$N \geq 256$; INT/FRAC coinciden con el registro CLKDIV.
DMA	Sin underrun durante al menos 10 minutos; contador de fallos igual a cero.
Frecuencia	Medición con contador calibrado; media a menos de 1 Hz del objetivo.
Espectro	Espurias del patrón dentro del límite definido para el transmisor.
USB	CDC/Audio/MSC permanecen enumerados durante arranque y operación.

7. Precisión absoluta y calibración

Frecuencia	1 Hz relativo	Estabilidad mínima de referencia
7,074 MHz	$1,414 \times 10^{-7}$	0,141363 ppm
14,074 MHz	$7,105 \times 10^{-8}$	0,071053 ppm
28,074 MHz	$3,562 \times 10^{-8}$	0,035620 ppm

Un cristal de 10 ppm puede introducir aproximadamente 70,74 Hz, 140,74 Hz y 280,74 Hz, respectivamente. Por lo tanto, la especificación física de ± 1 Hz exige calibración frente a una referencia mejor, compensación por temperatura o disciplina externa, por ejemplo un GPSDO o TCXO adecuado. Bresenham elimina cuantización media; no elimina error de referencia, ruido de fase ni deriva térmica. Sin embargo en todos los métodos de comunicación objetivo de éste proyecto importa mas el error relativo que el error absoluto.

Conclusiones

El análisis anterior requiere implementar primero el solver exhaustivo con las validaciones recomendadas en el PICO-SDK para a continuación registrar las mejores soluciones estáticas y confirmar los números de esta nota. Finalmente implementar el generador PIO+DMA con acumulador de fase, medir espurias y sólo entonces integrarlo al firmware USB. Dejar de reconfigurar PLL_SYS con TinyUSB montado; aplicar el PLL seleccionado antes de `TUDstart()` o reiniciar ante cambios que exijan otro PLL.

Si bien se estima como necesario una precisión de frecuencia de alrededor de 1 Hz no puede garantizarse físicamente sólo con búsqueda PLL y divisor PIO. Sí puede lograrse error medio de cuantización menor que 1 Hz —incluso cero teórico— mediante modulación temporal, pero la precisión absoluta requiere una referencia calibrada y validación espectral, sin embargo se comentó en la sección anterior que no es tan importante en los casos de uso.

Referencias

1. Raspberry Pi Ltd., *RP2040 Datasheet*, secciones de PLL, clocks y PIO.
2. Raspberry Pi Pico SDK, `hardware_clocks`: `check_sys_clock_hz()`, `set_sys_clock_pll()` y `clock_get_hz()`.
3. Raspberry Pi Pico SDK, `hardware_pio`: divisor de clock 16.8 y `pio_sm_set_clkdiv_int_frac8()`.
4. TinyUSB / Pico SDK: mantener `clk_usb` independiente y aplicar el PLL de sistema antes de enumerar el dispositivo.
5. Código del proyecto Firmware PixiePico, módulos `quad.c`, `quad.h` y checkpoint del 31-08-2026.